

An Agentic Evaluation Framework for AI-Generated Scientific Code in PETSc

Hong Zhang*, Barry Smith[†], Satish Balay*, Le Chen*, Murat Keceli*, Lois Curfman McInnes*, Junchao Zhang*

*Argonne National Laboratory, Lemont, IL, USA

{hongzhang, balay, lechen, keceli, mcinnes, jczhang}@anl.gov

[†]bsmith@petsc.dev

Abstract—While LLMs have accelerated scientific code generation, comprehensively evaluating generated code remains challenging. Many benchmarks emphasize functional correctness or task completion, which is insufficient for code built on production HPC libraries, where solver selection, API conventions, memory management, parallel awareness, and performance also matter. We introduce PETSCAGENT-BENCH, a multidimensional benchmark and agent-based framework for assessing whether AI-generated scientific code uses a production HPC library as an expert would. A tool-augmented evaluator compiles, executes, and measures code and combines deterministic checks with LLM-based assessments in a 14-evaluator pipeline spanning five categories: correctness, performance, code quality, algorithmic appropriateness, and library-specific conventions. A2A and MCP enable black-box evaluation of compatible coding agents. Across realistic PETSc problems, frontier models generate readable, well-structured code but struggle with correctness on challenging problems and with library-specific conventions even when code compiles and runs—limitations that conventional pass/fail evaluation does not capture.

Index Terms—Large language models, code generation, scientific computing, HPC, PETSc, agent-based evaluation, benchmarking

I. INTRODUCTION

Production scientific software is built on numerical libraries. Packages such as PETSc [1] and OpenFOAM [2] provide solvers, data structures, and interfaces developed over decades of research in numerical methods and parallel computing. Using them well is itself a demanding skill. A practitioner must know which components suit a given problem and how to compose them according to each library’s conventions.

Large language models (LLMs) are increasingly used to write such code, with demonstrated results in code generation [3], [4], code translation [5], and simulation workflow automation [6]; they are also now deployed as autonomous coding agents. The capability that matters for an LLM in this setting is using a library correctly. A capable model selects appropriate solvers and data structures for the problem, composes API calls in the prescribed patterns, and follows the library’s conventions for error handling and resource management.

Many code-generation benchmarks emphasize functional correctness or task completion (Section II). Many prompt an LLM to produce code and check it against test cases, often reducing the result to a pass/fail or pass@k score. HPC-oriented benchmarks may additionally incorporate compilation, parallel execution, performance, or numerical accuracy, but generally

do not assess whether a production numerical library is used as an expert would. For library-based code, *passing tests is necessary but not sufficient*. A representative case from our experiments makes this concrete. In our experiments, frontier models often generate PETSc code that compiles, runs, and produces the correct answer, yet use the legacy `CHKERRQ` macro rather than `PetscCall` for error handling, even though PETSc deprecated `CHKERRQ` in version 3.17 (2022). A pass/fail benchmark records such a submission as a complete success, while a PETSc developer would see at once that it does not follow current library conventions. What separates production-quality library code from merely working code is a set of properties these benchmarks do not measure. They include solver appropriateness, idiomatic API use, proper error handling, and efficient use of the library’s abstractions.

Measuring these properties calls for a different kind of evaluator. Two common evaluation approaches illustrate the complementary capabilities required: deterministic testing and LLM-based assessment. Neither is adequate on its own for the setting considered here. Static test scripts are reproducible and objective, but conventional correctness tests primarily establish whether code compiles, executes, and matches expected output. Such tests do not by themselves determine whether a call sequence is idiomatic or whether the chosen solver suits the problem. LLM-as-a-judge methods [7] assess style and semantics, but a text-only judge does not directly compile code against a library, run a binary in parallel, or measure execution time. Library-based scientific code requires both capabilities at once: execution and expert judgment.

We combine these capabilities in an agents-evaluating-agents framework. A tool-augmented evaluator agent compiles code against a library installation, executes it in a parallel environment, measures runtime performance, and applies both deterministic checks and LLM-based quality assessment. We realize this idea in PETSCAGENT-BENCH (Fig. 1), choosing PETSc (with over 6,000 functions spanning time integrators, nonlinear solvers, linear solvers, data management, and optimization) as the test case, because its breadth and complexity make the benchmark challenging and realistic.¹

¹PETSCAGENT-BENCH won third place in the Coding Agent track of the 2025-2026 AgentX-AgentBeats competition (<https://rdi.berkeley.edu/agentx-agentbeats.html>). The Green/Purple agent naming in this paper follows the AgentBeats convention.

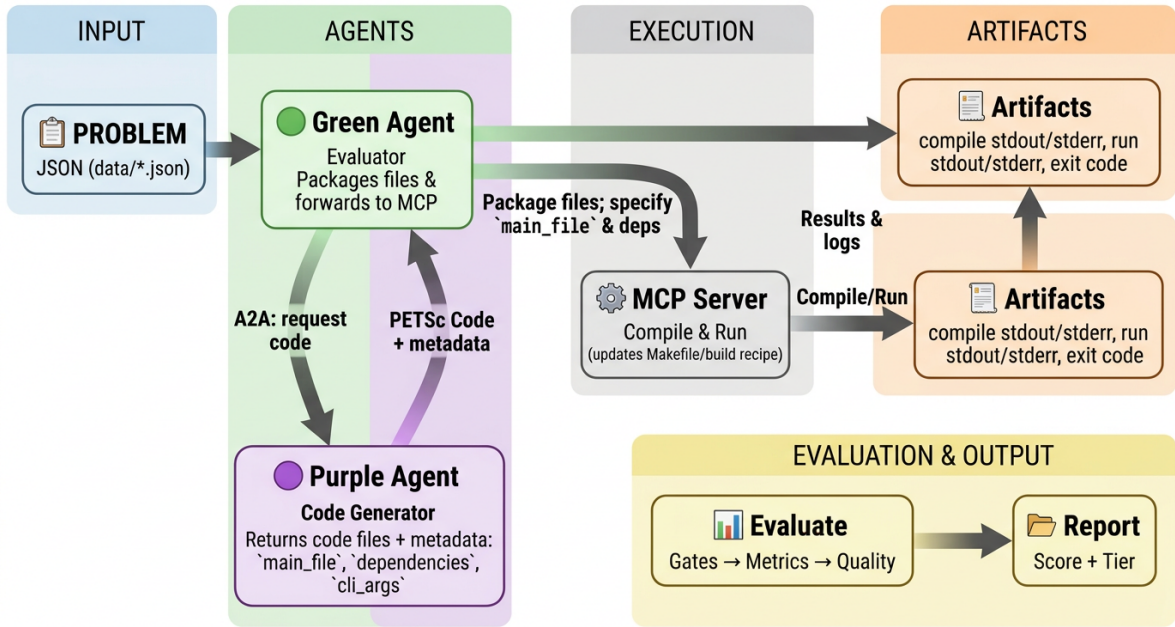


Fig. 1. Overview of the PETSCAGENT-BENCH architecture. The Green agent (evaluator) communicates with the Purple agent (model under test) through the A2A protocol and accesses compilation/execution tools through the MCP protocol.

Our contributions are:

- 1) **An agent-based evaluation framework for expert use of a production HPC library**, where a tool-augmented evaluator agent (Green agent) compiles, executes, and profiles code produced by a model-under-test agent (Purple agent) against a real HPC library installation, communicating through the Agent-to-Agent (A2A) [8] protocol and Model Context Protocol (MCP) [9] to enable black-box evaluation of interface-compatible coding agents without access to their internals.
- 2) **A multi-dimensional evaluation pipeline** with 14 evaluators across three stages (binary gates, quantitative metrics, and LLM-based quality assessments), aggregated into a confidence-weighted composite score.
- 3) **A curated PETSc benchmark suite** of six problems spanning key modules in PETSc and scientific domains.
- 4) **An empirical analysis** of coding agents backed by three frontier LLMs, revealing that current LLMs produce readable, well-structured code but struggle with correctness on challenging problems and with library-specific conventions even when the code compiles and runs, limitations that pass/fail benchmarks do not capture.

II. RELATED WORK

LLMs are increasingly being used in scientific software engineering, including code generation [10]–[12], HPC software development [13], and compiler validation [14]. Consequently, a growing body of benchmarks has emerged to evaluate LLM performance on programming and scientific computing tasks.

Code generation benchmarks. HumanEval [3] and MBPP [15] evaluate LLM-generated Python functions using

unit tests, while SWE-bench [16] and CodeContests [17] extend evaluation to repository-level tasks and algorithmic reasoning. ScienceAgentBench [18], AIInsteinBench [19], SciCode [20], MMSciCode [21], and SciCodePile [22] further benchmark models and agents on scientific coding tasks curated from research problems. These benchmarks primarily assess task completion or functional correctness. Most do not evaluate whether generated code uses an external scientific library idiomatically, selects suitable numerical algorithms, or follows library-specific parallel programming conventions.

HPC and scientific code generation. ParEval [4] benchmarks parallel code generation across six programming models and has been extended to repository-level tasks [23]. HPC-Coder-V2 [24] fine-tunes LLMs for parallel programming languages, while ChatHPC [25] presents foundations for a productive and trustworthy AI-assisted HPC ecosystem. Other work evaluates models such as DeepSeek [26] and studies performance optimization [27] and legacy code translation [28], [29]. Domain-specific benchmarks include FEM-Bench [30] for finite elements, CFDLLMBench [31] for computational fluid dynamics, and SciML Agents [32] for ODE solver generation. These efforts broaden evaluation to compiled and parallel languages, scientific problems, and, in some cases, numerical accuracy or performance. They nevertheless focus primarily on whether a generated implementation produces the expected result rather than whether it uses the abstractions and conventions of a mature numerical library as an expert would.

PETSc’s continuous-integration harness appropriately verifies examples by matching program output against stored reference files. It is designed to detect regressions, however, rather than to assess whether independently generated code

selects appropriate solvers. PETSCAGENT-BENCH addresses this complementary evaluation problem by testing not only whether generated code produces the correct result, but also whether it uses PETSc solvers, data structures, and programming conventions appropriately.

Agent architectures and evaluation. LLM-as-a-judge [7] uses a language model to score model outputs, and a growing body of software-engineering work adopts this paradigm to assess qualities that reference-based metrics miss, such as readability and adherence to best practices [33]. Although many such methods operate primarily on textual representations, recent agent-based evaluators augment LLM judgment with execution and other external evidence. CodeVisionary [34] decomposes task requirements, gathers contextual evidence through tools such as dynamic execution, unit testing, and static analysis, and uses multiple judges to produce fine-grained assessments. LLM4VV [14] similarly incorporates compilation and execution evidence into an agent-based validation pipeline for LLM-generated compiler tests. ProjDevBench [35] combines end-to-end execution with rule-based and LLM-assisted code review to evaluate complete software projects.

Multi-agent frameworks such as MOSAIC [36], PDE-Agent [37], and MooseAgent [6] use agents for code *generation* but evaluate the results with traditional test suites. Closest to our setting, our prior work developed retrieval-augmented coding assistants for PETSc [38] and agent-based workflows to improve PETSc usability and productivity [39]. These efforts focus on assisting PETSc use and development rather than at evaluating the quality of AI-generated PETSc code. A related concern is benchmark rigor, since agentic benchmarks can leak solutions or reward gaming instead of capability [40].

Relative to these efforts, PETSCAGENT-BENCH targets a distinct evaluation setting: expert use of a production HPC library. Its evaluator is itself a tool-augmented agent that compiles, executes, measures, and judges code through a multi-stage pipeline, communicating with the model under test via standardized protocols (A2A and MCP). The benchmark also reduces opportunities for gaming and contamination by separating problem specifications from reference-output checkers, reporting per-category scores, and altering tutorial-like problems. To our knowledge, PETSCAGENT-BENCH is the first agent-based benchmark designed specifically to evaluate whether AI-generated scientific HPC code uses a production numerical library in an expert, idiomatic, and parallel-aware manner.

III. AN AGENTIC EVALUATION FRAMEWORK FOR HPC LIBRARIES

Figure 1 shows the framework’s three components. The Green agent orchestrates evaluation, the Purple agent under test generates code, and the MCP server compiles and runs it against PETSc in a sandbox.

A. Design Principles

Evaluating library-based HPC code makes the evaluation infrastructure itself a substantial engineering problem. It must manage a PETSc installation, compile against it, run binaries under MPI, recover from compilation and runtime failures, and coordinate a sequence of evaluators. Bundling this into a single static harness would make the evaluator brittle and hard to extend.

An agentified architecture addresses this challenge by decoupling the model under test and the evaluator into independent agents and making the evaluator an autonomous agent, not a fixed test harness. In particular, our *agents-evaluating-agents* architecture stands on three principles: (1) role-based separation, where the evaluator and the model under test are independent agents that interact solely through A2A, and any agent speaking this protocol can be easily plugged into the evaluation infrastructure; (2) sandboxed execution, where generated code is compiled and run in a sandboxed PETSc/MPI environment through MCP tools; and (3) modular extensibility, where adding a benchmark problem requires only a JSON specification and adding an evaluator requires implementing a single Python class.

B. Component Overview

The system consists of three major components as shown in Fig. 1:

- **Green agent (evaluator).** The Green agent is an assessment manager that orchestrates the benchmark. It loads problem specifications from JSON files, dispatches each problem to a Purple agent through A2A messages, receives generated code and auxiliary information, invokes MCP tools for compilation and execution, runs the 14-evaluator pipeline (see Sec. IV), and prepares structured reports for artifact evaluation and leaderboard-style comparison.
- **Purple agent (model under test).** The Purple agent is an LLM-based code-generation agent that receives a natural-language problem description through A2A and returns one or more C source files together with metadata including the entry-point file, dependencies, and command-line arguments.
- **MCP server (tool provider).** The server provides a sandboxed execution environment exposing compilation and execution tools to the Green agent through MCP. It manages a custom PETSc installation, updates build recipes, compiles uploaded source files, runs the resulting binaries with MPI, and returns outputs (including errors and exit codes).

C. Protocol Usage

The framework relies on standardized protocols rather than a bespoke interface, so that any A2A-compatible agent can be evaluated as a black box and the growing ecosystem of MCP tools can be reused. The interaction between the Green agent and Purple agent is governed by the A2A protocol [8]. The Green agent creates a *task* containing the problem description

and the Purple agent responds with *artifacts*, such as source files and metadata. This decoupled setup allows the two agents to run on different machines, to sit behind firewalls, or to be hosted as cloud services.

The MCP [9] protocol governs the Green agent’s access to the compilation and execution environment. The MCP server exposes two primary tools: (1) `compile`, which builds the uploaded code against PETSc, and (2) `run`, which executes the binary with specified MPI ranks and command-line arguments. The reference implementation and public service endpoint are available at <https://github.com/petsc/petscagent-bench>.

IV. EVALUATION PIPELINE

The pipeline operationalizes the principle from Section I that for library-based code, passing tests is necessary but not sufficient. It is organized along two independent axes. The first axis is the *stage*, which controls execution order and cost. Evaluation proceeds through three sequential stages, *gates*, *metrics*, and *quality assessments*, in increasing order of expense. Gates are hard filters. If any gate fails, evaluation terminates with a zero score, so only code that compiles and runs reaches the more costly later stages. This enforces the necessary baseline before the pipeline spends effort measuring the dimensions that pass/fail benchmarks overlook.

The second axis is the *scoring category*, which determines how each evaluator contributes to the final score. Every evaluator belongs to one of five categories: **correctness** (syntactic, functional, semantic, and numerical), **performance** (execution time), **code** (readability, style, documentation), **appropriateness** (algorithm and solver selection), and **library-specific** (API conventions, error handling, parallel awareness). Stage and category are orthogonal. A single category can draw on evaluators from several stages, as correctness does through the compilation gate, the numerical-accuracy metric, and semantic checks. Table I lists all 14 evaluators with their stage, category, method, and confidence.

A. Stage 1: Gates

The four gates establish this necessary baseline. Evaluation continues only if the generated code passes all of them.

- **Compilation.** The C/CUDA/Kokkos source files compile against PETSc using PETSc’s Makefile system.
- **Execution.** The compiled binary runs to completion without crashes or errors.
- **Memory safety.** The submitted code must have no leaks or invalid memory accesses. This gate leverages runtime analysis tools (e.g., Valgrind) or PETSc’s own memory tracking features to detect memory issues.
- **API usage.** The code must follow correct PETSc patterns such as proper bracketing of `PetscInitialize` / `PetscFinalize` and using PETSc public header files.

B. Stage 2: Metrics

Each metric evaluator produces a numerical score in $[0, 1]$.

Numerical accuracy. The numerical output is compared against reference values from the problem specification. We

compute the relative error $\varepsilon = \|y_{\text{output}} - y_{\text{ref}}\| / \|y_{\text{ref}}\|$ and map it to a score with an exponential decay,

$$s_{\text{acc}} = \min(1, e^{-\varepsilon/\tau}), \quad (1)$$

where τ is a configurable tolerance defaulting to 10^{-3} .

Execution time. Runtime is scored by piecewise linear interpolation across four thresholds. The reported experiments use $t_1 = 0.5$ s, $t_2 = 1$ s, $t_3 = 2$ s, and $t_4 = 4$ s. Each problem may specify thresholds suited to its expected cost and target hardware. The score is 1.0 for $t \leq t_1$ and decays linearly through 0.8, 0.6, and 0.2 at t_2 , t_3 , and t_4 . For $t > t_4$ it falls off as $0.2 t_4 / t$. The current performance metric measures execution time for these benchmark problems on a fixed platform; it is not intended to measure parallel scalability.

C. Stage 3: Quality Assessments

The final and most expensive stage applies eight evaluators (Table I), each producing a score in $[0, 1]$, across the **code**, **appropriateness**, and **library-specific** categories. Six of the eight rely on LLM judgment, optionally augmented by static analysis or heuristics. The other two are library-specific and use deterministic checks. We reserve LLM-based evaluation for the subjective or semantic dimensions that deterministic techniques cannot capture, such as whether code is readable or whether a solver choice is appropriate, and use deterministic checks wherever a dimension can be verified directly. Evaluators without data dependencies run concurrently within a stage to reduce latency.

Because LLM-based evaluators can be sensitive to prompts and model versions, their scores should be interpreted as structured expert-assistance signals, not ground truth. We make each evaluator emit a rationale and confidence value, and the final report preserves the per-category scores so users can inspect whether a composite score is driven by deterministic failures, numerical accuracy, or subjective quality judgments. In the current benchmark release, we therefore use composite scores primarily for ranking and diagnosis within a fixed evaluator configuration. Validating LLM-based quality scores against independent PETSc expert review is an important next step.

D. Score Aggregation

Individual evaluator scores are aggregated in two steps to produce the final composite score in $[0, 100]$.

Step 1: Category scores. Each evaluator j in category c gives a score s_j in $[0, 1]$ with confidence α_j . The scores are weighted by confidence and aggregated into a single category score S_c for each of the five categories:

$$S_c = \frac{\sum_{j \in c} \alpha_j s_j}{\sum_{j \in c} \alpha_j}. \quad (2)$$

Confidence is a variable returned by the LLM for LLM-based evaluators and a fixed value for deterministic evaluators.

TABLE I
THE 14 EVALUATORS IN THE PETSCAGENT-BENCH PIPELINE.

#	Evaluator	Stage	Category	Method	Conf.
1	Compilation	Gate	Correctness (syntactic)	Deterministic	1.0
2	Execution	Gate	Correctness (functional)	Deterministic	1.0
3	Memory Safety	Gate	Correctness (semantic)	Runtime	0.7–1.0*
4	API Usage	Gate	Library-specific	Deterministic	1.0
5	Numerical Accuracy	Metric	Correctness (numerical)	Runtime	1.0
6	Execution Time	Metric	Performance	Runtime	1.0
7	Readability	Quality	Code	LLM	LLM
8	Code Style	Quality	Code	LLM + static	LLM
9	Documentation	Quality	Code	LLM	LLM
10	Algorithm Approp.	Quality	Appropriateness	LLM	LLM
11	Solver Choice	Quality	Appropriateness	LLM + heuristic	LLM
12	Best Practices	Quality	Library-specific	LLM + patterns	LLM
13	Error Handling	Quality	Library-specific	Deterministic	1.0
14	Parallel Awareness	Quality	Library-specific	Deterministic	0.8

*1.0 with Valgrind; 0.7 without.

Step 2: Composite score. Scores across the categories are rescaled to 0–100 and then combined with configurable weights w_c to produce the final composite score:

$$S_{\text{composite}} = \sum_c w_c S_c. \quad (3)$$

The default weights are correctness 0.35, performance 0.15, code 0.15, library-specific 0.20, appropriateness 0.15. Correctness and library-specific quality receive the largest weights because failures in either dimension can invalidate production scientific code. The weights define the benchmark policy and remain configurable for other applications. The composite score is not intended to replace the category breakdown. For example, a submission can obtain a moderate score by passing gates and running quickly while still failing PETSc-specific best practices. Conversely, readable code can score poorly if numerical accuracy or execution fails. We therefore report category-level scores and gate failures alongside the composite score in all analyses.

V. BENCHMARK SUITE

We curate a suite of six problems (Table II) designed to probe whether coding agents can use PETSc as an expert would. The selection balances several goals. The problems exercise distinct PETSc subsystems, spanning time integration (TS), optimization (Tao), linear solvers and mesh management (KSP, DM), and distributed data (Vec), and they cover a range of numerical methods from ODE integration through hyperbolic and elliptic PDEs to nonlinear optimization and parallel data management, each demanding a different mode of reasoning. They also span a difficulty range from easy to hard, so the suite discriminates among models rather than saturating or defeating all of them. Where a problem resembles an existing PETSc tutorial, we modify it so that memorized solutions do not transfer, as detailed at the end of this section. We intend these six problems as an initial benchmark and expect the suite to expand across more of PETSc’s functionality.

Each problem is specified as a JSON file with a natural-language description of the governing equations, boundary conditions, and requirements, together with test cases giving command-line arguments, MPI rank counts, time limits, and reference outputs. The descriptions are deliberately under-specified. We withhold problem characteristics that an expert would have to infer, such as the stiffness of an ODE, the appropriate solver, or the specific PETSc API, so that success requires reasoning about the problem instead of transcribing a known recipe. This design lets the suite test library-selection skill rather than recall.

The problems fall into three difficulty tiers that probe increasing levels of this skill. The easy problems, Rosenbrock optimization, 1D advection, and Vec/MPI scatter, test whether an agent can wire up the correct PETSc interface for a well-posed task, including Tao callback signatures, upwind discretization with periodic boundaries, and distributed vector communication through VecScatter. The medium problem, Robertson ODE, further requires recognizing a numerical property, stiffness, and responding with an implicit integrator and appropriate tolerances. The hard problems, Darcy flow and 2D Navier–Stokes, demand composing multiple subsystems, finite-element assembly over heterogeneous coefficients for the former and coupled multi-field nonlinear solves on a staggered grid for the latter. Reference outputs come from hand-written PETSc implementations and are checked with problem-specific tolerances, while the API and quality evaluators judge whether the generated code follows PETSc conventions beyond matching the final number.

PETSc’s source repository is public and almost certainly part of LLM training data. To mitigate data contamination [41], we intentionally alter initial conditions, boundary conditions, or discretizations if similar problems exist in PETSc tutorials. These changes reduce the likelihood that models can simply reproduce memorized tutorial solutions, but they do not eliminate all contamination risk. For example, PETSc’s tutorial suite solves Darcy flow with finite differences. Our specification requires a finite-element formulation

TABLE II
BENCHMARK PROBLEMS IN PETSCAGENT-BENCH. MODULE INDICATES THE PRIMARY PETSC COMPONENT EXERCISED; DIFFICULTY IS ASSESSED ON A THREE-LEVEL SCALE.

Problem	Module	Difficulty	Domain	Key Challenge
Robertson ODE	TS	Medium	Stiff ODE system	Stiffness, adaptive stepping
1D Advection	TS	Easy	Hyperbolic PDE	Upwind discretization, periodic BCs
Rosenbrock Opt.	Tao	Easy	Nonlinear optimization	Gradient computation, convergence
Darcy Flow	KSP, DM	Hard	Elliptic PDE (FEM)	FE assembly, heterog. coefficients
2D Navier–Stokes	TS, SNES	Hard	Incompressible flow	Staggered grid, implicit coupling
Vec/MPI Scatter	Vec	Easy	Data management	MPI communication, VecScatter

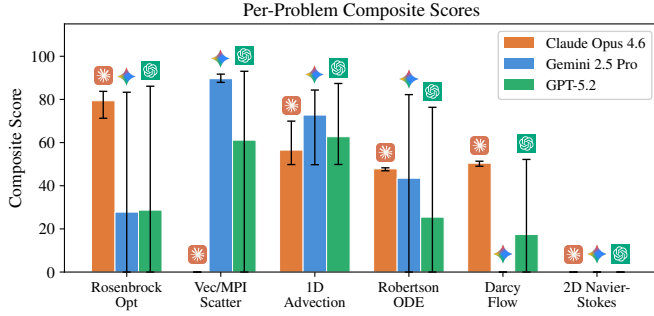


Fig. 2. Per-problem composite scores. Bars show the mean over three runs per model, counting gate failures as zero; error bars span the minimum and maximum of those runs.

instead, forcing the model to use higher-level abstractions such as PETSc’s DMPLEX module for mesh management and element assembly.

VI. EVALUATION RESULTS AND ANALYSIS

We evaluate three frontier coding agents on the benchmark. Across the board, the models produce readable, well-structured code yet struggle with correctness on hard problems and with PETSc-specific conventions throughout, dimensions that pass/fail benchmarks would not surface. The following analysis breaks this pattern down by problem, by category, and by failure mode.

A. Evaluation Settings

Each agent uses a frontier LLM (Claude Opus 4.6, Gemini 2.5 Pro, or GPT-5.2) and is given only one shot to generate code to solve each problem. Each test is repeated three times under identical conditions to account for variability in LLM outputs. The Green agent uses a single fixed model, GPT-5.2, for all LLM-based quality evaluators, so that every submission is judged under identical criteria and quality scores remain comparable across models. All models are run at temperature 0 to aid reproducibility, although hosted APIs may still introduce implementation-level nondeterminism. We also pin the completion-token limit to 32,000, because the API default of 4,096 truncates and thus invalidates the longest solutions. Each Purple agent is evaluated in a one-shot setting. It receives the natural-language specification once and gets no compiler errors or test feedback for iterative repair. The protocol measures unaided first-attempt performance from the

problem specification alone. Retrieval, planning, and repair can be evaluated as separate agent configurations. The Green agent compiles each submission against a fixed PETSc/MPI environment. The experiments used PETSc 3.25.3 with GCC 11.4.0 and MPICH 5.0.0 on an 8-core Intel Core i7-11700KF workstation with 32 GB RAM; complete environment details are provided in the artifact appendix. The Green agent records source files, compilation commands, stdout/stderr, exit codes, metric outputs, evaluator rationales, and final scores. The artifact package (available at <https://github.com/petsc/petscagent-bench>) contains the problem JSON files, reference-output checkers, evaluator prompts and rubrics, configuration files, and scripts required to replay cached submissions and rerun the benchmark with new A2A-compatible agents.

B. Overall Performance

All three models achieve only modest average composite scores. Claude Opus 4.6 and Gemini 2.5 Pro are effectively tied at **39.0** and **38.9**, with GPT-5.2 trailing at **32.6**. Of the 54 total attempts, 25 score zero from gate failures while 13 exceed 80, reflecting a benchmark that separates clear successes from clear failures. These averages summarize the 54 observed attempts: Claude and Gemini differ by 0.1 points, and GPT-5.2 trails them by 6.3–6.4 points. All three models become less reliable on problems that require both numerical reasoning and detailed PETSc knowledge.

Although GPT-5.2 is both a model under test and the quality judge, we do not observe an obvious self-preference effect in the aggregate results. The judge scores only the subjective quality dimensions, which carry a minority of the composite weight, while the heaviest category, correctness at 0.35, and performance are measured deterministically. Using the same judge for every submission keeps the rubric fixed across models, but it does not eliminate the possibility of model- or style-specific judge bias. GPT-5.2 places last overall despite judging its own code, which suggests that any such bias is not large enough to dominate the reported ranking; we nevertheless treat judge-model bias as a limitation.

C. Per-Problem Analysis

A clear difficulty gradient can be seen in Figure 2. When gates are passed, the easier problems score well, with Rosenbrock averaging 79–86 and Vec/MPI Scatter 90–92, but neither is passed reliably by every model. Claude fails all three

Vec/MPI attempts, while Gemini and GPT-5.2 each fail Rosenbrock in two of three runs. Medium problems exhibit greater variability. 1D Advection averages 57–73 and passes every gate, while Robertson ODE scores 48–76 when it runs but clears gates in only six of nine attempts. The hard problems, Darcy Flow and 2D Navier–Stokes, defeat nearly all attempts. On Darcy Flow, Claude passes all three runs at an average of 51 and GPT-5.2 passes once at 52, while Gemini fails every attempt. No model passes 2D Navier–Stokes.

All three models fail on the Navier–Stokes problem, but for distinct reasons. Gemini 2.5 Pro fails at compilation in all three runs, passing incorrect argument types to `DMStagCreate2d`, for example supplying `NULL` where PETSc expects a `PetscInt` stencil width, indicating a fundamental misunderstanding of the DMStag API. Claude Opus 4.6 and GPT-5.2 both compile successfully but then crash at runtime. In five of the six runs, PETSc complains about “Matrix is missing diagonal entries,” indicating that the Jacobian assembly omits some grid points. One of GPT-5.2’s attempts fails with `DIVERGED_NONLINEAR_SOLVE`, implying a problematic nonlinear solver configuration. None of the three frontier models solves this case in any of its three one-shot attempts.

D. Gate Failure Analysis

Gate pass rates are **67%** for Claude, **50%** for Gemini, and **44%** for GPT-5.2. Among submissions that reach successful execution, all pass the remaining memory-safety and API-usage gates. All 29 such submissions used the standard-error memory screen at confidence 0.7. This locates the primary gate bottlenecks before the later quality stages: first in generating code that compiles against PETSc, and then in avoiding runtime failures on the hardest problems. Gate reliability differs substantially across models, even though their composite scores are close. For all three agents, however, every gate failure is a compilation or runtime failure.

E. Category-Level Analysis

Figure 3 shows broadly similar profiles across models. Scores are averaged over gate-passed attempts only. **Performance** is uniformly high at 86–100, reflecting fast code execution across all problems. **Code**, 79–83, and **appropriateness**, 75–81, are also strong, indicating that the agents tend to generate readable, well-structured code with reasonable choices of numerical method and solvers. **Correctness** varies widely by model and problem, from 33 to 76, as some problems, particularly stiff ones, are highly sensitive to solver parameters and numerical stability. **Library-specific** scores are uniformly weak, never exceeding 67 for any model, driven by systematic failures in PETSc-specific error handling and best-practice compliance.

F. Qualitative Findings

Our multidimensional evaluation highlights several informative cases. For the Vec/MPI Scatter problem, Claude triggers

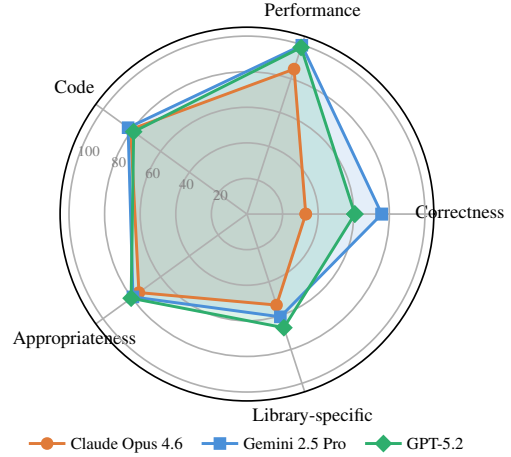


Fig. 3. Category-level scores on a 0–100 scale, averaged over gate-passed attempts only (Claude: $n = 12$; Gemini: $n = 9$; GPT-5.2: $n = 8$, of 18 attempts each).

TABLE III
TWO JUDGES SCORING THE SAME GENERATED PROGRAMS. A DASH MARKS A GATE FAILURE, SCORED ZERO BY EITHER JUDGE; THE COMPOSITE COVERS ALL SIX PROBLEMS. 2D NAVIER–STOKES IS OMITTED BECAUSE ALL THREE CACHED SUBMISSIONS FAIL A GATE AND THEREFORE RECEIVE ZERO UNDER EITHER JUDGE.

Model	Judge	Test problem					Comp.
		Rosen.	Robert.	Vec	Advec.	Darcy	
Claude	GPT	83.8	47.6	–	70.1	51.4	42.1
	Claude	84.7	49.0	–	69.5	50.3	42.3
Gemini	GPT	87.6	49.3	87.9	49.7	–	45.8
	Claude	87.7	49.3	86.0	49.6	–	45.4
GPT-5.2	GPT	84.7	78.1	93.4	82.5	–	56.4
	Claude	86.6	77.2	88.9	82.2	–	55.8

segmentation faults in three attempts despite otherwise high-quality code. On the stiff Robertson ODE problem, Gemini 2.5 Pro produces clean, well-structured code but poor numerical accuracy due to loose integrator tolerances it selects. Error-handling scores are especially low, exactly 0.30 in 20 of 29 gate-passed attempts, because most models do not wrap PETSc calls in `PetscCall()`. This suggests that current LLMs have learned PETSc usage from older versions (pre-3.17) that used the legacy `CHKERRQ()` style.

G. Robustness to the Choice of Judge

To test whether the quality scores depend on the choice of judge, we rerun the quality evaluators with a second judge, Claude Opus 4.6. We cache the code generated in one run and replay it against the second judge, so that both judges see byte-identical programs. They differ by **1.1** points on average over the 12 gate-passed attempts shown in Table III. The largest disagreement is 4.5 points, and neither judge scores consistently higher. The composite scores agree within **0.6** points for every model, far less than the several-point shift seen

when the same model and judge are rerun on freshly generated code. In this experiment, generation variability exceeds the score variation introduced by substituting the second judge.

VII. DISCUSSION

Why library-based evaluation matters. The dimensions that separate high from low scores, including error-handling conventions, solver selection, and resource-management patterns, are inherently library-specific and do not arise in from-scratch implementations. By targeting PETSc, PETSCAGENT-BENCH measures the skill that production HPC code actually demands, composing scalable components correctly instead of reimplementing known algorithms.

Our results show that models fail at this skill in two distinct ways, and the distinction matters because the two have very different remedies. One failure mode appears to reflect knowledge staleness. The widespread use of the deprecated `CHKERRQ()` macro instead of `PetscCall()` reflects training data that predates PETSc 3.17, suggesting a problem that retrieval of current documentation may mitigate. Failures in assembling Jacobians for finite-element methods and in matching integrator tolerances to problem stiffness reflect gaps in numerical reasoning, not stale facts, and these will not be closed by fresher documentation alone.

Implications for agent design. These two failure modes call for different interventions. Knowledge-staleness failures are the more tractable. Retrieval augmentation over current PETSc documentation could supply the idioms models are missing, and the prevalence of compilation failures suggests that multi-turn agents with compile-fix loops may improve pass rates, since compiler diagnostics are usually informative enough to guide corrections. Numerical-reasoning failures are harder and unlikely to yield to context or retries alone. The framework can measure gains from retrieval, planning, and repair under the same tasks and scoring criteria.

Limitations. The benchmark does not yet cover PETSc’s full breadth, such as GPU kernels, multiphysics, advanced preconditioners, performance portability, and large-scale distributed-memory scaling, but can be extended with additional JSON problem specifications and evaluator classes. The present runtime metric compares fixed test instances on one workstation. Multi-node scaling studies require parameterized problem sizes and rank counts. In addition, LLM-based evaluators may vary across runs due to the inherent stochasticity of LLMs, though this is mitigated by temperature 0, fixed prompts, preserved rationales, and confidence weighting. An expert-audit study could compare general-purpose and domain-adapted judges against independent PETSc maintainers on a fixed set of submissions. The current results are also limited to one-shot generation. Multi-turn agents that can read compiler errors and repair submissions may obtain different results. Finally, modifying tutorial-like problems reduces but does not eliminate training-data contamination risk.

Responsible release and benchmark use. The benchmark does not release models or human-subject data, but benchmark

artifacts can still shape model-development incentives. To reduce leaderboard gaming, the artifact separates public problem specifications from reference-output checkers and preserves per-category reports instead of only a single score. The main intended use is diagnostic evaluation of scientific-code agents, not certification that generated code is safe for production scientific computing. Users should still review generated code manually before using it in research or engineering workflows.

VIII. CONCLUSION AND FUTURE WORK

We introduced PETSCAGENT-BENCH, a multidimensional benchmark and agent-based evaluation framework for AI-generated scientific code that targets *library-based* HPC programming rather than from-scratch implementations. Its evaluator is a tool-augmented agent that builds and runs each submission against a real PETSc installation and scores it along multiple dimensions, communicating with the model under test via standardized protocols. This design supports black-box evaluation that goes beyond traditional pass/fail benchmarking.

Our evaluation of Claude Opus 4.6, Gemini 2.5 Pro, and GPT-5.2 shows that frontier LLMs generate readable, well-structured scientific code but fall short on correctness and library-specific conventions, dimensions that traditional pass/fail evaluation does not capture. Replaying identical programs past a second judge moves the quality scores by about a point, suggesting that the main findings are not sensitive to the choice between these two judges. We are extending the suite to GPU test problems and additional PETSc subsystems. More broadly, as LLMs take on a growing share of library-based scientific programming, evaluating whether they use libraries the way an expert would becomes essential, and PETSCAGENT-BENCH is a step toward making that skill measurable.

ACKNOWLEDGMENTS

This material is based upon work partially supported by Laboratory Directed Research and Development (LDRD) funding from Argonne National Laboratory, provided by the Director, Office of Science, of the U.S. Department of Energy (DOE) under Contract No. DE-AC02-06CH11357. This work was also partially supported by the U.S. DOE, Office of Science, Office of Advanced Scientific Computing Research, by the Competitive Portfolios for Advanced Scientific Computing Research Program, by the Scientific Discovery through Advanced Computing (SciDAC) Program through the FASTMath Institute, and by the U.S. DOE Office of Science Distinguished Scientist Fellows Program. This research used resources of the Argonne Leadership Computing Facility, a U.S. Department of Energy (DOE) Office of Science user facility at Argonne National Laboratory.

REFERENCES

- [1] S. Balay, S. Abhyankar, M. F. Adams *et al.*, “PETSc/TAO users manual,” Argonne National Laboratory, Tech. Rep. ANL-21/39 – Revision 3.25, 2026. [Online]. Available: <https://petsc.org/release/manual/manual.pdf>
- [2] C. Greenshields, *OpenFOAM v13 User Guide*. London, UK: The OpenFOAM Foundation, 2025. [Online]. Available: <https://doc.cfd.direct/openfoam/user-guide-v13>

- [3] M. Chen, J. Tworek, H. Jun *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [4] D. Nichols, J. H. Davis, Z. Xie, A. Rajaram, and A. Bhatele, “Can large language models write parallel code?” in *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, 2024.
- [5] L. Chen, N. Xu, W. Chen, B. Lei, P.-H. Lin, D. Zhou, R. Thakur, C. Ding, A. Jannesari, and C. Liao, “Beyond code pairs: Dialogue-based data generation for llm code translation,” in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026, pp. 33 781–33 803.
- [6] T. Zhang, Z. Liu, Y. Xin, and Y. Jiao, “MooseAgent: A LLM based multi-agent framework for automating MOOSE simulation,” *arXiv preprint arXiv:2504.08621*, 2025.
- [7] L. Zheng, W.-L. Chiang, Y. Sheng *et al.*, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023.
- [8] Google, “Agent-to-agent (A2A) protocol specification,” 2025.
- [9] Anthropic, “Model context protocol (MCP) specification,” 2024.
- [10] L. Chen, A. Bhattacharjee, N. Ahmed, N. Hasabnis, G. Oren, V. Vo, and A. Jannesari, “OMPGPT: A generative pre-trained transformer model for OpenMP,” in *European Conference on Parallel Processing*. Springer, 2024, pp. 121–134.
- [11] D. Nichols, A. Marathe, H. Menon, T. Gamblin, and A. Bhatele, “HPC-Coder: Modeling parallel programs using large language models,” in *ISC High Performance 2024 Research Paper Proceedings (39th International Conference)*. Prometheus GmbH, 2024, pp. 1–12.
- [12] P. Valero-Lara, A. Huante, M. A. Lail, W. F. Godoy, K. Teranishi, P. Balaprakash, and J. S. Vetter, “Comparing Llama-2 and GPT-3 LLMs for HPC kernels generation,” 2023. [Online]. Available: <https://arxiv.org/abs/2309.07103>
- [13] W. F. Godoy, P. Valero-Lara, K. Teranishi, P. Balaprakash, and J. S. Vetter, “Large language model evaluation for high-performance computing software development,” *Concurrency and Computation: Practice and Experience*, vol. 36, no. 26, p. e8269, 2024.
- [14] C. Munley, A. Jarmusch, and S. Chandrasekaran, “Llm4vv: Developing llm-driven testsuite for compiler validation,” *Future Generation Computer Systems*, vol. 160, pp. 1–13, 2024.
- [15] J. Austin, A. Odena, M. Nye *et al.*, “Program synthesis with large language models,” *arXiv preprint arXiv:2108.07732*, 2021.
- [16] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.
- [17] Y. Li, D. Choi, J. Chung *et al.*, “Competition-level code generation with AlphaCode,” *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [18] Z. Chen, S. Chen, Y. Ning, Q. Zhang *et al.*, “ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery,” in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.
- [19] T. Duston, S. Xin, Y. Sun, D. Zan *et al.*, “AIInsteinBench: Benchmarking coding agents on scientific repositories,” *arXiv preprint arXiv:2512.21373*, 2025.
- [20] M. Tian, L. Gao, S. D. Zhang, X. Chen, C. Fan, X. Guo, R. Haas, P. Ji, K. Krongchon, Y. Li, S. Liu, D. Luo, Y. Ma, H. Tong, K. Trinh, C. Tian, Z. Wang, B. Wu, Y. Xiong, S. Yin, M. Zhu, K. Lieret, Y. Lu, G. Liu, Y. Du, T. Tao, O. Press, J. Callan, E. Huerta, and H. Peng, “SciCode: A research coding benchmark curated by scientists,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.
- [21] X. Xia, Z. Yang, A. Cohan, and Y. Zhao, “MMSciCode: Real-world evaluation of multilingual multi-discipline scientific research coding,” in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026, pp. 33 981–33 999.
- [22] W. Sun, Y. Fan, Y. Chen, G. Tan, J. Shi, Y. Yidi, S. L. Wong, J. Pan, and D. Lo, “SciCodePile: A 128GB corpus and executable benchmark for challenging scientific code generation,” *arXiv preprint arXiv:2607.19104*, 2026.
- [23] J. H. Davis, D. Nichols, I. Khillan, and A. Bhatele, “ParEval-Repo: A benchmark suite for evaluating LLMs with repository-level HPC translation tasks,” in *Proceedings of the 54th International Conference on Parallel Processing (ICPP)*, 2025, pp. 94–103.
- [24] A. Chaturvedi, D. Nichols, S. Singh, and A. Bhatele, “HPC-Coder-V2: Studying code LLMs across low-resource parallel languages,” in *Proceedings of the International Conference on High Performance Computing (ISC)*, 2025, pp. 1–14.
- [25] P. Valero-Lara, A. R. Young, J. S. Vetter, Z. Jin, S. Pophale, M. A. H. Monil, K. Teranishi, and W. F. Godoy, “ChatHPC: Building the foundations for a productive and trustworthy AI-assisted HPC ecosystem,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2025.
- [26] N. Nader, P. Diehl, S. Brandt, and H. Kaiser, “LLM & HPC: Benchmarking DeepSeek’s performance in high-performance computing tasks,” in *Proceedings of the International Conference on High Performance Computing (ISC)*, 2025, pp. 626–638.
- [27] B. Cui, T. Ramesh, O. Hernandez, and K. Zhou, “Do large language models understand performance optimization?” *arXiv preprint arXiv:2503.13772*, 2025.
- [28] S. Gupta, K. Kamalakkannan, M. Moraru, G. Shipman, and P. Diehl, “From legacy Fortran to portable Kokkos: An autonomous agentic AI workflow,” *arXiv:2509.12443*, 2025.
- [29] M. T. Dearing, Y. Tao, X. Wu, Z. Lan, and V. Taylor, “Leveraging LLMs to automate energy-aware refactoring of parallel scientific codes,” *arXiv preprint arXiv:2505.02184*, 2025.
- [30] S. Mohammadzadeh, E. Hamdi, J. Shor, and E. Lejeune, “FEM-Bench: A structured scientific reasoning benchmark for evaluating code-generating LLMs,” *arXiv preprint arXiv:2512.20732*, 2025.
- [31] N. Somasekharan, L. Yue, Y. Cao, W. Li, P. Emami *et al.*, “CFD-LLM-Bench: A benchmark suite for evaluating large language models in computational fluid dynamics,” *arXiv preprint arXiv:2509.20374*, 2025.
- [32] S. Gaonkar, X. Zheng, H. Xi, R. Tiwari, K. Keutzer, D. Morozov, M. W. Mahoney, and A. Gholami, “SciML agents: Write the solver, not the solution,” *arXiv preprint arXiv:2509.09936*, 2025.
- [33] J. He, J. Shi, T. Y. Zhuo, C. Treude, J. Sun, Z. Xing, X. Du, and D. Lo, “LLM-as-a-judge for software engineering: Literature review, vision, and the road ahead,” *ACM Transactions on Software Engineering and Methodology*, 2026, to appear.
- [34] X. Wang, P. Gao, C. Peng, R. Hu, and C. Gao, “CodeVisionary: An agent-based framework for evaluating large language models in code generation,” *arXiv preprint arXiv:2504.13472*, 2025.
- [35] P. Lu, S. Zhang, Y. Hou, L. Ye, C. Huang, Z. Chen, J. Zeng, H. Jiang, P. Liu, Y. Wang, and M.-H. Yang, “ProjDevBench: Benchmarking AI coding agents on end-to-end project development,” *arXiv preprint arXiv:2602.01655*, 2026.
- [36] S. Raghavan and T. Mallick, “MOSAIC: Multi-agent orchestration for task-intelligent scientific coding,” *arXiv preprint arXiv:2510.08804*, 2025.
- [37] J. Liu, R. Zhu, J. Xu, K. Ding, X.-Y. Zhang, G. Meng, and C.-L. Liu, “PDE-Agent: A toolchain-augmented multi-agent framework for PDE solving,” *arXiv preprint arXiv:2512.16214*, 2025.
- [38] B. Smith, J. Zhang, H. Zhang *et al.*, “AI assistants to enhance and exploit the PETSc knowledge base,” in *54th International Conference on Parallel Processing Companion (ICPP)*. ACM, 2025, doi:[10.1145/3750720.3757281](https://doi.org/10.1145/3750720.3757281).
- [39] B. Smith, H. Zhang, J. Zhang, S. Balay, L. Chen, M. Keçeli, and L. C. McInnes, “Improving usability and productivity of PETSc with agent-based workflows,” in *PEARC ’26: Proceedings of the Practice and Experience in Advanced Research Computing 2026: Resilient Roots + Empowered Communities*. ACM, 07 2026, article 38, doi:[10.1145/3785462.3815882](https://doi.org/10.1145/3785462.3815882).
- [40] Y. Zhu, T. Jin, Y. Pruksachatkun, A. Zhang, S. Liu, S. Cui, S. Kapoor, S. Longpre, K. Meng, R. Weiss, F. Barez, R. Gupta, J. Dhamala, J. Merizian, M. Giulianelli, H. Coppock, C. Ududec, J. Sekhon, J. Steinhardt, A. Kellermann, S. Schwettmann, M. Zaharia, I. Stoica, P. Liang, and D. Kang, “Establishing best practices for building rigorous agentic benchmarks,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2025.
- [41] A. Jacovi, A. Caciularu, O. Goldman, and Y. Goldberg, “Stop uploading test data in plain text: Practical strategies for mitigating data contamination by evaluation benchmarks,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.